Beleganforderung

12.10.2026

1. Projekt

- Bau einer kleinen, aber vollständigen Web-Anweundung:
  - Frontend
  - REST-API
  - Datenbank
  - Absicherung
- Dokumentation der technischen Entscheidungen

Themenvorschläge

- Rezeptsammlung: Rezepte mit Zutaten, Kategorien und Suche
- Bücherei/Mediensammlung: Titel, Autoren, Genre, Ausleihstatus
- Notiz-/Aufgabenverwaltung: Notizen mit Tags, Erledigt-Status, Filter

Oder

- Eigener Themenvorschlag
- Muss folgende Merkmale erfüllen: mindestens 2 verknüpfte Entitäten, CRUD, Suche/Filter, Nutzer-Anmeldung

2. Rahmenbedingungen

Rahmen	Vorgabe
Umfang	**MVP-Logik** — bewusst klein, aber vollständig und korrekt
Teamgröße	Einzelarbeit oder **Gruppenarbeit (max. 3 Personen)** — bei Gruppen: einzelne Belegbestandteile markieren
Zeitraum	Abgabe am **08.02.2026**
Frontend	Möglichst Vanilla HTML/CSS/JS
Backend	Kleiner Webserver: Java mit JDK HttpServer. Frameworks nur nach Absprache mit erweitertem Belegumfang zu Funktionalitäten
Datenbank	Relational: SQLite, PostgreSQL oder MySQL
Build-Tools	Maven/Gradle im Backend erlaubt
Abgabe	Git-Repository, Nextcloud der Hochschule o.a. Projekt, Beleg und ein Video mit der Ausführung müssen auf jeden Fall bei mir ankommen

3. Technologie-Vorgaben

Frontend

- HTML, CSS, JavaScript
- Kein React, Vue, Svelte, Angular …

Backend

- Webserver mit JDK HttpServer
- Kein Springboot, Quarkus, Micronaut
- Validierung
- Passwort-Hashing mit bcrypt/argon2 (sollte nie selbst gebaut werden)
- Datenbankzugriff mit rohem SQL Treiber (mysql2, psycopg, JDBC) oder ORM (Sequelize, Prisma, SQLAlchemy, Hibernate)
- Maven oder Gradle können genutzt werden

4. Anforderungen

4.1. Muss

Frontend

- Semantisches HTML: (Landmarks, Überschriften-Hierarchie, sinnvolle Elemente)
- Externes Stylesheet, Flexbox oder Grid
- JavaScript: `fetch` gegen die eigene API, `async/await`, Fehlerbehandlung
- Formulare mit Clientseitiger Validierung

Backend

- REST-API mit mindestens GET (Liste + Detail), POST, PUT/PATCH, DELETE
- Korrekte Status-Codes
- CRUD komplett for die Hauptentitäten
- Mindestens eine Verknüpfung zwischen zwei Entitäten (z.B.: Rezept und Zutaten)
- Server-Seite Inputvalidierung
- Sessions oder Tokens für das Anmelden

Datenbank

- Relationale DB mit mind. 2 Tabellen + Foreign-Key Beziehung
- Zugriff über ORM oder rohes SQL

Sicherheit

- SQL-Injection unmöglich
- XSS-Behandlung
- Hashing von Passwörtern
- CORS korrekt konfiguriert

Extra

- Video, dass die Anwendung demonstriert

4.2. Kann für Extrapunkte

- Pagination oder Limit bei Listenansicht
- Einheitliche Fehlerformate
- Such- und Filterfunktion serverseitig
- Tests

5. Was wird nicht bewertet

- Schönheit des Designs: Funktionalitäten sind hier bedeutend wichtiger